

# Human Emotion Classification

## CS 613 Final Project

Lev Saunders, Neel Rakeshbhai Patel, Yabo Detchou,  
Anna Grace, Kaarthik Rohan Kumar

March 2026

## 1 Abstract

This project presents a machine learning approach to human emotion classification using facial expression recognition on real-world images. Leveraging the large-scale AffectNet dataset, which contains diverse and unconstrained facial images, we develop a classification pipeline that incorporates data cleaning, preprocessing, dimensionality reduction using Principal Component Analysis (PCA), and multiple classification models including Support Vector Machines (SVM), k-Nearest Neighbors (KNN), and Random Forests. Images are standardized through grayscale conversion and resizing, and extensive filtering is applied to remove corrupted, duplicate, and low-quality samples. Experimental results show that while the models are capable of identifying certain emotions—particularly happiness—overall performance remains limited, with accuracy below 33% across all classifiers. These findings highlight the challenges of multi-class emotion recognition in unconstrained environments and suggest the need for more advanced feature extraction and modeling techniques.

## 2 Introduction

Facial expression recognition is an important problem in computer vision and machine learning because facial expressions provide valuable information about human emotions and behavior. Systems that can automatically detect and classify facial expressions have applications in a wide range of fields, including human–computer interaction, mental health monitoring, and social robotics. By enabling computers to interpret human emotions from visual information, facial expression recognition systems can support technologies that interact with people in more natural and meaningful ways.

Despite significant progress in facial recognition technologies, accurately recognizing facial expressions in real-world environments remains challenging. Many early models were developed using highly controlled laboratory datasets where lighting conditions, camera angles, and facial visibility were carefully

managed. In contrast, real-world images often contain poor lighting, blur, occlusions, and significant variation in facial appearance. These factors can make it difficult for models trained on controlled datasets to perform well when applied to real-world data. In addition, human emotional expression is complex and involves a wide range of subtle facial movements, making it necessary for models to distinguish between multiple emotion categories rather than simple expressions alone.

The goal of this project is to develop a machine learning system capable of classifying facial expressions across ten distinct emotion categories. These emotions include common expressions such as happiness, sadness, anger, and other affective states. By training models to recognize a broader range of emotions, the project aims to create a system that more accurately reflects the complexity of real human emotional communication. The project focuses on building this system using real-world image data rather than controlled laboratory images in order to better evaluate model performance in practical scenarios.

To support this work, the project uses the AffectNet dataset, a large-scale facial expression dataset containing approximately 450,000 human-labeled images collected from the web. Because these images were gathered “in the wild,” they include substantial variation in lighting conditions, facial poses, image resolution, and background environments. This diversity makes the dataset more challenging but also more realistic for evaluating machine learning approaches to facial expression recognition.

The project explores several machine learning methods for facial expression classification including nonlinear Support Vector Machines (SVMs) and k-Nearest Neighbors (KNN), both of which have been shown to perform well in previous facial expression recognition research. Additionally we employ an ensemble machine learning model, Random Forest classification, towards the same classification task. We also make use of Principal Component Analysis (PCA) for reducing the dimensionality of our input data, which has been demonstrated in related studies.

Before model training, several preprocessing steps are applied to prepare the dataset. These steps include converting images to grayscale and resizing images to standardized dimensions. Proper preprocessing helps reduce noise in the dataset, standardizes the input data, and improves the ability of machine learning models to detect meaningful patterns in facial features.

By combining careful preprocessing, dimensionality reduction, and multiple classification models, this project aims to develop a practical system for facial expression recognition that performs effectively on real-world image data. The results of this work contribute to understanding how machine learning techniques can be applied to complex emotion recognition tasks and how different modeling strategies perform when applied to challenging real-world datasets.

### 3 Related Works

Many studies have been done on using machine learning for human emotion classification. Relevant to our project are papers which cover relatively simple models which we have covered over the course of CS 613. In *Whitehill et al.*, researchers demonstrate accurate smile detection in real world photography conditions using Support Vector Machines, paired with Gabor Energy Filters used in preprocessing. In *Cornejo et al.*, researchers demonstrate both SVM and K-NN models for general emotion classification with resistance to facial occlusion. They additionally demonstrate increased performance using PCA as a dimensionality reduction step.

### 4 Dataset

#### 4.1 Affectnet

The dataset used in this project is AffectNet, a large-scale facial expression dataset commonly used in emotion recognition research. The dataset contains images of human faces annotated with emotion labels and bounding box coordinates indicating the location of the face within the image. AffectNet was selected because it provides a large and diverse set of facial images captured under varying lighting conditions, poses, and backgrounds, making it suitable for training robust computer vision models.

In this project, the dataset is obtained through Kaggle using the kaggle-hub API, which allows automated dataset download and extraction within the project environment. The import script retrieves the dataset and organizes its contents into separate directories for images and label files. After downloading, all image files are copied into a centralized images directory while corresponding label files are placed in a labels directory. Each label file contains annotation information corresponding to a specific image. The import process ensures that the dataset is organized consistently and removes temporary cache files created during the download process.

#### 4.2 Raw Data Structure

The raw dataset consists of two primary components: image files and annotation files. The image files are stored in either JPG or PNG format and contain images of human faces collected from a variety of sources. The annotation files are text files containing label and bounding box information for each corresponding image.

Each annotation file includes several values that describe the labeled face within the image. These values include the emotion label assigned to the image, the center coordinates of the bounding box surrounding the face, and the width and height of the bounding box. The bounding box coordinates are normalized relative to the image dimensions, which is a common format used in object detection datasets.

Before processing, the dataset is stored in a raw data directory containing two folders: one for images and one for labels. This structure ensures that each image can easily be matched with its corresponding annotation file during the data cleaning and processing stages.

### 4.3 Data Cleaning and Quality Filtering

Raw image datasets often contain corrupted files, duplicate images, or images with poor visual quality. These issues can negatively affect machine learning models by introducing noise into the training data. To address these problems, a comprehensive cleaning procedure is implemented before the dataset is used for modeling.

The first step of the cleaning process is detecting and removing corrupt image files. Each image is opened and verified using the Python Imaging Library (PIL). If the image cannot be successfully opened or verified, it is considered corrupt and removed from the dataset. This step ensures that all remaining images can be processed without errors.

Next, duplicate images are identified and removed. To detect duplicates, the image data is converted into a numerical array and an MD5 hash is generated from the pixel values. If two images produce identical hash values, they are considered duplicates and only one copy is retained. Removing duplicates prevents redundant data from biasing the training process.

The cleaning process also removes images that are blurry. Blur detection is performed using a Laplacian-based filtering method that measures the amount of edge variation in the image. Images with low variance after applying the filter are classified as blurry and removed because they contain insufficient visual detail for reliable facial recognition.

Another quality check identifies images that are significantly overexposed or underexposed. The mean pixel intensity of each grayscale image is calculated, and images with values outside an acceptable intensity range are discarded. Images with extreme brightness or darkness often lack meaningful visual features and can degrade model performance.

## 5 Classification Pipeline

Our classification pipeline, shown in Fig. 1, takes input images, preprocesses them, runs them through a PCA stage for dimensionality reduction, and then feeds the output data into our SVM, KNN, and Random Forest models for classification.

### 5.1 Preprocessing

After filtering, all remaining images are standardized to ensure consistency across the dataset. Each image is converted to grayscale and resized to 96 by 96 pixels.

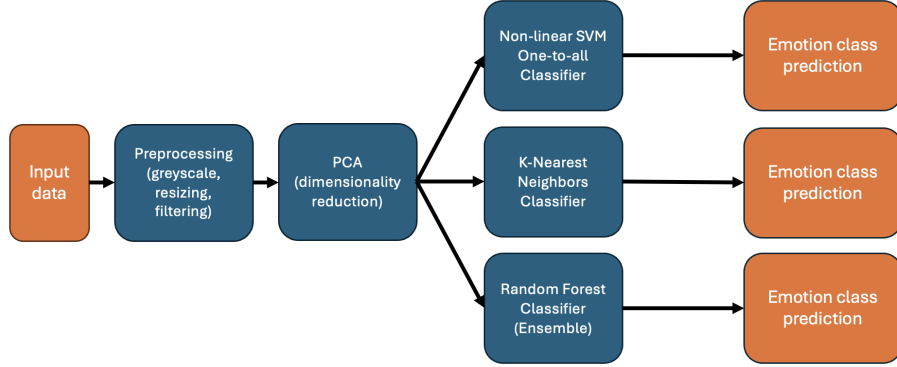


Figure 1: classification pipeline

Standardizing the image size is important because most machine learning models require fixed input dimensions. Converting images to grayscale reduces computational complexity while preserving the structural features of the face that are most relevant for expression recognition.

#### 5.1.1 Dataset Split

Once the images are cleaned and standardized, the dataset is divided into three subsets: a training set, a validation set, and a test set. 70% of the images are assigned to the training set, 15% to the validation set, and the remaining 15% to the test set. The dataset is randomly shuffled before splitting to ensure that each subset contains a representative sample of the full dataset. The training set is used to train machine learning models, the validation set is used to tune model parameters and evaluate intermediate performance, and the test set is reserved for final model evaluation.

#### 5.1.2 Annotation File Generation

To simplify downstream model training and analysis, the project generates a structured CSV annotation file containing the image filename, emotion label, bounding box coordinates, and the dataset split associated with each image.

The annotation file allows the dataset to be easily loaded using data analysis and machine learning libraries such as pandas. By consolidating all metadata into a single file, the pipeline simplifies dataset management and ensures that experiments can be reproduced consistently.

### 5.2 PCA

We utilize principal component analysis for dimensionality reduction. Calculated without any machine learning libraries, we compute the principal components of our training dataset via singular value decomposition.

### 5.3 SVM

For our first classification model, we use an one-to-all support vector machine classifier. Here we train a single binary SVM classifier for each output class, and compute the output prediction as the SVM classifier with the greatest confidence. We perform cross-validation, sweeping over linear, polynomial, and radial basis function kernels while also varying the parameters in order to find the best configuration for the current dataset.

### 5.4 KNN

For the second classification model, we implemented a K-nearest neighbors architecture, which takes the feature-reduced image data as input and produces an output emotion prediction. For the model's  $k$  value, we do a hyperparameter search and choose the value used by highest performing model.

### 5.5 Random Forest Classifier

For our ensemble classifier we have chosen a Random Forest model which is a commonly used model for image classification tasks. again, we perform hyperparameter tuning, searching for the most accurate model over a range of values for the number of trees used and the maximum depth of the trees.

## 6 Results

### 6.1 PCA image reconstruction

Our PCA implementation is able to successfully reconstruct images with an arbitrary number of principal components. Looking at Fig. 2, you can see the successful reconstruction of both a test set image and a validation set image with a varying number of principal components. When choosing a number of principal components to use in our classification pipeline, we selected 150, as with this number of components the size of the image is drastically reduced, but emotions are still recognizable to the human eye.

### 6.2 SVM

When training our one-to-all SVM, our cross-validation showed the best performance using a radial basis function kernel with a gamma value of  $\gamma = 0.02$ .

### 6.3 KNN

When training our KNN Classifier, we ran hyperparameter tuning where we tested  $k$ -values  $k \in [1, 99]$  with a step size of 2. From this we had the best accuracy with  $k = 61$

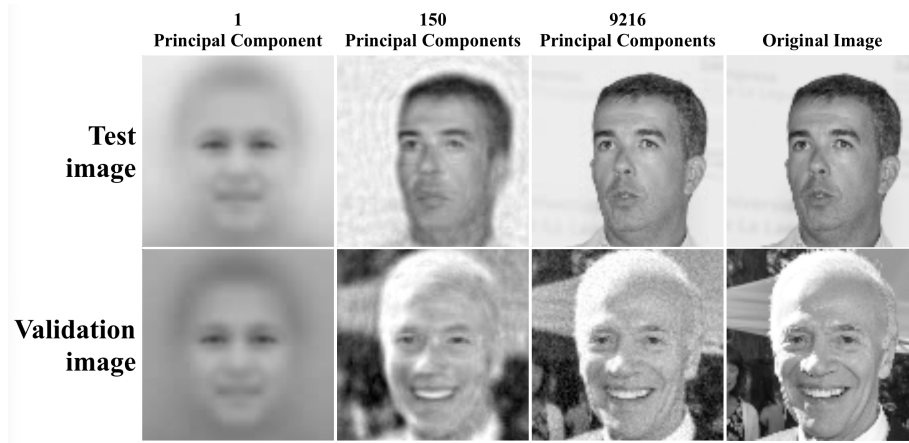


Figure 2: PCA reconstruction of a test and train image with an increasing number of principal components used

## 6.4 Random Forest

When training our Random Classifier, we ran hyperparameter tuning where we tested a range of values for the number and max depth of the trees that make up the random forest classifier. The best performing parameters from this testing were from a classifier made up of 60 trees with a max depth of 5.

## 6.5 Performance

Looking at the gross accuracy, F1 Score and SMAPE of our three classifiers seen in Table 1, we observe poor overall performance. No model is able to classify emotions from the test set with more than 33% accuracy. The best performing is our SVM, with the KNN and Random Forest following in that order.

In order to get a better picture of the performance of our models, we look at per-class performance, seen in Table 2. Here we can see that across all classes, there is poor precision, recall, and F1 score. Notably, while still poor, all 3 models score significantly better with the 'Happy' output class than with any other class.

All models were trained with an equal balance of all classes, so this difference in performance likely indicates that the "happy" images are more separable than the other classes in the input data space.

## 7 Discussion

When comparing our models' performances to those in *Whitehill et al.* and *Cornejo et al.*, there are several points we observe. the most prominent departure between the methodology employed by these two papers an our own work

| Model         | Test Accuracy | Test F1 Score | Test SMAPE |
|---------------|---------------|---------------|------------|
| SVM           | 33%           | 32.37%        | 69.88%     |
| KNN           | 21.37%        | 21.11%        | 75.19%     |
| Random Forest | 21.92%        | 20.78%        | 82.11%     |

Table 1: Performance Metrics per Classifier

| Class    | Precision |       |               | Recall |      |               | F1    |       |               |
|----------|-----------|-------|---------------|--------|------|---------------|-------|-------|---------------|
|          | SVM       | KNN   | Random Forest | SVM    | KNN  | Random Forest | SVM   | KNN   | Random Forest |
| Anger    | 0.356     | 0.375 | 0.103         | 0.37   | 0.09 | 0.1           | 0.363 | 0.145 | 0.102         |
| Contempt | 0.337     | 0.169 | 0.107         | 0.3    | 0.15 | 0.08          | 0.317 | 0.159 | 0.091         |
| Disgust  | 0.242     | 0.143 | 0.171         | 0.23   | 0.13 | 0.18          | 0.236 | 0.136 | 0.176         |
| Fear     | 0.260     | 0.205 | 0.197         | 0.25   | 0.23 | 0.15          | 0.255 | 0.217 | 0.170         |
| Happy    | 0.576     | 0.614 | 0.422         | 0.72   | 0.43 | 0.57          | 0.640 | 0.506 | 0.485         |
| Neutral  | 0.276     | 0.157 | 0.152         | 0.29   | 0.29 | 0.14          | 0.283 | 0.204 | 0.146         |
| Sad      | 0.270     | 0.140 | 0.169         | 0.27   | 0.07 | 0.15          | 0.270 | 0.093 | 0.159         |
| Surprise | 0.244     | 0.179 | 0.198         | 0.21   | 0.32 | 0.26          | 0.226 | 0.229 | 0.225         |

Table 2: Per-Class Performance Metrics

is that they both used Gabor energy filters, which model the texture filtering done by primate visual cortexes and compute a much smaller feature vector.

Another point of interest from the results of our models is their relatively strong performance of the 'Happy' emotion class compared to all other classes. Since there was an even class split in the training data, it is possible that this discrepancy is due to the 'happy' images being more separable from the other classes in the feature space.

Another note is that while *Whitehill et al.* does demonstrate human facial emotion classification with PCA and SVMs, they are doing binary classification whereas we are attempting multi-class classification. This makes it more difficult to use this research as a point of comparison.

## 8 Future Work

Future work should primarily focus on increasing model performance for the primary task of emotion classification. This could be potentially done by implementing Gabor energy filtering, as is done in *Whitehill et al.* and *Cornejo et al.*.

Another area of study could be analyzing the performance of our model if it were to be trained to be able to discriminate between fewer output classes. For instance, grouping 'similar' emotions such as *fear*, *disgust*, *contempt*, and *anger*, as well as groups such as *happy* and *surprise* has the potential to improve model performance.

Our SVM model uses an all-to-one classification scheme, but future work could focus on swapping this with a one-to-one scheme could help improve discrimination of harder to distinguish emotions.

Finally, it would likely be useful to seek out existing research that has been done using the AffectNet dataset to see if there are methods that we have not yet considered for improving emotion classification.

## 9 References

1. Kaggle Data Source
2. Fard, A. P., Hosseini, M. M., Sweeny, T. D., & Mahoor, M. H. (2024). AffectNet+: A Database for Enhancing Facial Expression Recognition with Soft-Labels. arXiv preprint arXiv:2410.22506.
3. J. Whitehill, G. Littlewort, I. Fasel, M. Bartlett and J. Movellan, "Toward Practical Smile Detection," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 31, no. 11, pp. 2106-2111, Nov. 2009, <https://doi.org/10.1109/TPAMI.2009.42>
4. Cornejo, J.Y.R., Pedrini, H., Flórez-Revuelta, F. (2015). Facial Expression Recognition with Occlusions Based on Geometric Representation. In: Pardo, A., Kittler, J. (eds) Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications. CIARP 2015. Lecture Notes in Computer Science, vol 9423. Springer, Cham. [https://doi.org/10.1007/978-3-319-25751-8\\_32](https://doi.org/10.1007/978-3-319-25751-8_32)